

DAT12-2

Programming in Python

Automation, logic, and computation

Read a file, process it, write the result.

Albert School — 40 hours

Contents

1	The development environment	2
2	Core data types	2
3	Control flow	2
4	Functions	3
5	File I/O and exceptions	3
6	Third-party libraries	4
7	Debugging	4
8	Virtual environments	4
9	Object-oriented programming	4
10	Data processing	5
11	APIs	5

1 The development environment

Python programs are written as text files with a `.py` extension and run from the command line.

Definition 1 VS Code (or an equivalent editor) is used to write and edit Python source files. A program is run from the command line with `python script.py`.

Definition 2 `pip` is the tool that installs Python packages. A package is installed with `pip install <name>`.

A working environment therefore consists of an editor for writing code and a command line from which scripts are run and packages installed.

2 Core data types

Definition 3 Python's core data types are:

- `int`: a whole number.
- `float`: a number with a fractional part.
- `str`: a string of text.
- `list`: an ordered, mutable sequence of values, written `[a, b, c]`.
- `dict`: a mapping from keys to values, written `{k: v}`.
- `tuple`: an ordered, immutable sequence of values, written `(a, b, c)`.
- `bool`: a truth value, `True` or `False`.

The types differ in what they hold and in how they may be used. `int` and `float` are numbers and support arithmetic. `str` holds text. A `list` may be changed after creation; a `tuple` may not. A `dict` retrieves a value by its key rather than by position. A `bool` records a yes/no condition. Each type is chosen for the kind of value it holds: numbers as `int` or `float`, text as `str`, an ordered collection as a `list` or `tuple`, a key-value association as a `dict`.

3 Control flow

Definition 4 A **conditional** chooses which statements to run based on a condition:

```

if condition:
    ...
elif other_condition:
    ...
else:
    ...

```

Definition 5 A **loop** repeats statements. A **for** loop iterates over the items of a sequence; a **while** loop repeats as long as a condition holds.

Control flow determines the order in which statements execute. A conditional runs a block only when its condition is true. A **for** loop runs its body once per item of a collection; a **while** loop runs its body until its condition becomes false.

4 Functions

Definition 6 A **function** is a named, reusable block of code. It is defined with **def**, may declare parameters with type annotations, and returns a value with **return**:

```

def add(a: int, b: int) -> int:
    """Return the sum of a and b."""
    return a + b

```

Definition 7 A **docstring** is the string written on the first line of a function body; it documents what the function does. **Scope** is the region of a program in which a name is defined: names assigned inside a function are local to it.

A function is called by writing its name followed by arguments in parentheses, `add(2, 3)`. Type annotations state the expected types of the parameters and the return value. The docstring records the function's purpose.

5 File I/O and exceptions

Definition 8 A file is opened with **open**. Used as a **context manager** with **with**, the file is closed automatically when the block ends:

```

with open("data.txt", "r") as f:
    contents = f.read()

```

The mode "r" reads, "w" writes, "a" appends.

Definition 9 **Exception handling** runs code that may fail inside a **try** block and handles the error in an **except** block:

```

try:
    ...
except SomeError:
    ...

```

Reading a file retrieves its contents; writing creates or overwrites a file with new contents. The `with` statement guarantees the file is closed. An operation that may raise an error (such as opening a file that does not exist) is placed in a `try` block so the error can be caught and handled rather than stopping the program.

6 Third-party libraries

Definition 10 A **third-party library** is code written by others and distributed as a package. It is installed with `pip install <name>` and made available in a script with `import <name>`.

Using a library means installing it with `pip`, importing it, and calling its functions as documented. The library's documentation states what functions it provides and how to call them.

7 Debugging

Definition 11 **Debugging** is the systematic process of locating and correcting errors in a program.

Debugging proceeds by reading the error message Python prints (which names the error type and the line where it occurred), by inserting `print` statements to observe the values of variables as the program runs, and by reasoning about the state of the program — which variables hold which values at each point — to identify where behaviour diverges from intent.

8 Virtual environments

Definition 12 A **virtual environment** is an isolated Python installation for a single project, keeping its packages separate from other projects and from the system Python.

Definition 13 A `requirements.txt` file lists the packages a project depends on. The environment is reproduced by installing from it with `pip install -r requirements.txt`.

A virtual environment isolates the packages and Python version a project uses, so that projects do not interfere with one another. It is created from scratch, its packages are installed into it, and it is reproduced elsewhere from a `requirements.txt` file listing those packages.

9 Object-oriented programming

Definition 14 A **class** defines a type of object by its attributes (data) and methods (functions that act on the data). An **object** is an instance of a class:

```
class Account:
    def __init__(self, balance: float):
        self.balance = balance

    def deposit(self, amount: float) -> None:
        self.balance += amount
```

A class is defined with `class`. The `__init__` method sets up a new object's attributes. An object is created by calling the class, `Account(100)`. Attributes hold the object's data; methods are functions called on the object, `account.deposit(50)`, that read or change its attributes.

10 Data processing

Definition 15 A CSV file stores tabular data as rows of comma-separated values. The standard library `csv` module reads and writes CSV files.

A data-processing script reads data from a CSV file, processes it (filtering, transforming, or aggregating the rows), and writes the result to a new file. This combines file I/O, control flow, and the core data types into a single program that turns one file into another.

11 APIs

Definition 16 An **API** (Application Programming Interface) is a defined way for one program to request data or services from another, commonly over HTTP.

Definition 17 An **HTTP GET request** retrieves data from a web address. The `requests` library performs one with `requests.get(url)`. A **JSON** response is parsed into Python data (dicts and lists) with the response's `.json()` method:

```
import requests
response = requests.get(url)
data = response.json()
```

An API is called by sending an HTTP GET request to its address with `requests`. The response carries data, often in JSON format, which is parsed into Python dictionaries and lists and then processed like any other data.

Exercises

1. Set up a Python development environment: install Python, create a virtual environment, and run a script that prints a message from the command line.
2. For each core data type (`int`, `float`, `str`, `list`, `dict`, `tuple`, `bool`), create a value of that type and state what distinguishes it from the others.
3. Write a script that uses an `if/elif/else` conditional and both a `for` and a `while` loop to process a collection of values.
4. Write, document with a docstring, and call a function with typed parameters and a return value.
5. Write a script that opens a file with a context manager, reads it, and writes output to another file, handling a possible error with `try/except`.
6. Install a third-party library with `pip`, import it into a script, and call one of its functions using its documentation.
7. Take a program that raises an error and debug it by reading the error message, adding print statements, and reasoning about the program state.
8. Create a virtual environment from scratch, install packages into it, record them in a `requirements.txt` file, and reproduce the environment from that file.
9. Define a class with attributes and methods, instantiate it, and call its methods.

- 10.** Write a script that reads data from a CSV file, processes it, and writes the result to a new file.
- 11.** Make an HTTP GET request to a public API with `requests`, parse the JSON response, and print a value from it.